PIMPRI-CHINCHWAD EDUCATION TRUST'S

Pimpri-Chinchwad College of Engineering

(An Autonomous Institute) • Affiliated to Savitribai Phule Pune University (SPPU) • ISO 21001:2018 Certified by TUV

ASSIGNMENT NO 7

Name:	**Siddhesh Parkar**	**Course:**	Design and Analysis of Algorithm
PRN:	**124B1F021**	**Class:**	TY IT A
Topic:	**Graph Coloring (University Exam Timetable Scheduling)**	**Batch:**	A1

1. Problem Statement

University Exam Timetable Scheduling: A university faces scheduling conflicts due to overlapping student course enrollments. No student enrolled in multiple courses can have exams scheduled in the same time slot. Furthermore, available examination classrooms per slot are physically constrained. Model this problem as an undirected conflict graph and implement **Graph Coloring using Backtracking** to determine the minimum number of conflict-free examination slots (chromatic number) and assign classrooms.

2. Course Objectives

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

- Apply Backtracking strategies to solve various problems.
- Classify P, NP, NP-complete, and NP-Hard problems.

4. Theory

The exam scheduling problem is modeled using Graph Theory:

- **Vertices (V):** Academic courses requiring examination.
- **Edges (E):** Pairwise conflicts where at least one student is enrolled in both courses.
- **Colors:** Examination time slots. The minimum number of colors needed without assigning the same color to adjacent vertices is the graph's **Chromatic Number $\chi(G)$**.
- **Classroom Allocation:** Courses assigned to the same time slot must not exceed the count of available examination rooms.

Comparison of Graph Coloring Techniques:

Feature	Greedy Coloring	Welsh-Powell	DSATUR	Backtracking (Exact)
Strategy	First available color	Decreasing degree order	Dynamic saturation degree	Systematic depth-first search
Optimality	Suboptimal	Heuristic	Near-optimal	**Guaranteed Minimum Colors**
Time Complexity	O(V + E)	O(V log V + E)	$O(V^2 + E)$	$O(k^V)$ worst case

5. Algorithm & Pseudo Code

```
procedure graphColoring(v, n, k, graph, colors):
    if v == n then return true
    for color ← 1 to k do
        if isSafe(v, color, graph, colors) then
            colors[v] ← color
            if graphColoring(v + 1, n, k, graph, colors) then return true
            colors[v] ← 0   // Backtrack
        end if
    end for
    return false
end procedure

function findMinimumSlots(graph):
    for k ← 1 to n do
        if graphColoring(0, n, k, graph, colors) then return k
    end for
end function
```

6. Program Source Code (C++)

```
/*
 * Student Name: Siddhesh Parkar
 * PRN: 124B1F021
 * GitHub Repository: https://github.com/siddhesh160606/DAA_lab
 * Assignment 7: University Exam Timetable Scheduling Using Graph Coloring and Backtracking
 */

#include <iostream>
#include <vector>
#include <string>

using namespace std;

bool isSafe(int v, int color,
            const vector<vector<int>>& graph,
            const vector<int>& colors)
{
    for (int u : graph[v]) {
        if (colors[u] == color)
            return false;
    }
    return true;
}

bool graphColoring(int v, int n, int k,
                   const vector<vector<int>>& graph,
                   vector<int>& colors)
{
    if (v == n)
        return true;

    for (int color = 1; color <= k; color++) {
        if (isSafe(v, color, graph, colors)) {
            colors[v] = color;
            if (graphColoring(v + 1, n, k, graph, colors))
                return true;
            colors[v] = 0;
        }
    }
    return false;
}

int findMinimumSlots(const vector<vector<int>>& graph,
                     vector<int>& result)
{
    int n = graph.size();
    for (int k = 1; k <= n; k++) {
        vector<int> colors(n, 0);
        if (graphColoring(0, n, k, graph, colors)) {
            result = colors;
            return k;
        }
    }
    return n;
}

int main() {
    cout << "Student Name : Siddhesh Parkar\n";
    cout << "PRN          : 124B1F021\n";
    cout << "GitHub Link  : https://github.com/siddhesh160606/DAA_lab\n";
    cout << "------------------------------------------------------\n";

    int n, rooms, edges, pairs;
    cout << "Enter number of courses: ";
    cin >> n;
```

```
    cout << "Enter number of rooms: ";
    cin >> rooms;

    vector<string> course(n);
    cout << "\nEnter course names:\n";
    for (int i = 0; i < n; i++) {
        cout << "Course " << i + 1 << ": ";
        cin >> course[i];
    }

    vector<vector<int>> graph(n);
    cout << "\nEnter number of conflict edges: ";
    cin >> edges;

    cout << "Enter conflict edges (course numbers):\n";
    cout << "Example: 1 2 means Course 1 conflicts with Course 2\n";
    for (int i = 0; i < edges; i++) {
        cout << "Edge " << i + 1 << ": ";
        int u, v;
        cin >> u >> v;
        u--;
        v--;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    cout << "\nEnter number of course pairs having common students: ";
    cin >> pairs;

    cout << "Enter course pairs having common students:\n";
    cout << "Example: 1 3 means courses 1 and 3 have common students\n";
    for (int i = 0; i < pairs; i++) {
        cout << "Pair " << i + 1 << ": ";
        int u, v;
        cin >> u >> v;
        u--;
        v--;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<int> colors;
    int slots = findMinimumSlots(graph, colors);

    cout << "\n================================\n";
    cout << " EXAM TIMETABLE\n";
    cout << "================================\n";
    cout << "Minimum exam slots: " << slots << "\n";

    for (int slot = 1; slot <= slots; slot++) {
        cout << "\nSlot " << slot << ":\n";
        int room = 1;
        for (int i = 0; i < n; i++) {
            if (colors[i] == slot) {
                if (room <= rooms) {
                    cout << "Room " << room << " -> " << course[i] << "\n";
                    room++;
                }
                else {
                    cout << "No room available -> " << course[i] << "\n";
                }
            }
        }
    }
```

```
    return 0;
}
```

7. Sample Execution Output

```
============================================================
Student Name: Siddhesh Parkar
PRN         : 124B1F021
GitHub Repo : https://github.com/siddhesh160606/DAA_lab
Assignment  : 7 - University Exam Timetable Scheduling Using Graph Coloring and Backtracking
============================================================

Enter number of courses: 5
Enter number of rooms: 3

Enter course names:
Course 1: maths
Course 2: physics
Course 3: chemistry
Course 4: cs
Course 5: english

Enter number of conflict edges: 5
Enter conflict edges (course numbers):
Example: 1 2 means Course 1 conflicts with Course 2
Edge 1: 1 2
Edge 2: 1 3
Edge 3: 1 5
Edge 4: 2 4
Edge 5: 3 4

Enter number of course pairs having common students: 2
Enter course pairs having common students:
Example: 1 3 means courses 1 and 3 have common students
Pair 1: 1 2
Pair 2: 2 4

================================
 EXAM TIMETABLE
================================
Minimum exam slots: 2

Slot 1:
Room 1 -> maths
Room 2 -> cs

Slot 2:
Room 1 -> physics
Room 2 -> chemistry
Room 3 -> english
```

8. Conclusion

Graph Coloring via Backtracking guarantees an optimal, conflict-free examination timetable. By iteratively searching for the smallest feasible chromatic number *k*, the algorithm minimizes student exam fatigue, eliminates overlapping schedules, and pairs compatible course exams to available university lecture halls efficiently.

GitHub Repository Link

Repository Link: https://github.com/siddhesh160606/DAA_lab